

GalenoSv

Guía para explicar y defender la arquitectura, el flujo funcional y las decisiones técnicas del sistema de gestión odontológica.

Enfoque: Jakarta EE 11, JSF, PrimeFaces, JPA, EJB, PostgreSQL y Open Liberty.

Fecha de corte: 17 de septiembre de 2026.

Propósito: exposición del proyecto hasta la fase implementada.

Cómo usar esta guía

El documento está pensado para dos situaciones: presentar el proyecto de forma ordenada y responder preguntas sobre por qué se tomó una decisión concreta. No hace falta memorizar todo. Conviene dominar el recorrido de una petición y los casos destacados en los recuadros.

Idea principal para abrir la exposición:

GalenoSv adapta el patrón trabajado por el ingeniero a una clínica odontológica. Se conserva la separación entre persistencia, acceso a datos, modelo de vista y páginas JSF, pero cada ejemplo se ajusta a las relaciones reales de la base de datos.

Orden recomendado de exposición

1. Objetivo y alcance actual.
2. Tecnologías y despliegue.
3. Arquitectura por capas y recorrido de una petición.
4. Modelo de datos y flujo de las páginas.
5. CRUD genérico con `DefaultDAO` y `AbstractModel`.
6. Caso especial `ExamenTipoExamen`.
7. Interfaz, convertidores e internacionalización.
8. Pruebas, estado real y próximos pasos.

1. Qué es GalenoSv

GalenoSv es una aplicación web para organizar información clínica y odontológica. En la fase actual permite administrar catálogos, clínicas, personas, documentos, medios de contacto, roles, procedimientos, pasos clínicos, consultas, órdenes de examen y resultados.

La aplicación no se planteó como una colección de formularios aislados. Las páginas siguen las relaciones de la base de datos: primero se preparan catálogos y definiciones; después se registran personas; finalmente se ejecuta el flujo de atención.

Estado cuantificable

Elemento	Cantidad	Interpretación
Entidades JPA	20	Representan las 20 tablas trabajadas en esta fase.
DAO concretos	20	Uno por entidad, además de <code>DefaultDAO</code> y la interfaz común.
Modelos JSF	20	Diecinueve respaldan páginas visibles y uno administra un detalle interno.
Páginas XHTML	19	<code>ExamenTipoExamen</code> se integra dentro de <code>Examen.xhtml</code> .
Pruebas automatizadas	14	Actualmente cubren comportamiento de <code>AbstractModel</code> y DAO seleccionados.

2. Tecnologías y función de cada una

Tecnología	Función dentro de GalenoSv	Decisión relevante
Java 21	Lenguaje y plataforma de compilación.	Maven compila explícitamente con <code>source</code> y <code>target 21</code> .
Jakarta EE 11	APIs de JSF, CDI, EJB, JPA, validación y transacciones.	La dependencia tiene alcance <code>provided</code> porque Liberty suministra las APIs.
JSF / Facelets	Enlaza formularios XHTML con modelos Java.	Las páginas usan la plantilla común de <code>WEB-INF</code> .
PrimeFaces 15.0.17 Jakarta	Tablas, menús, diálogos, autocompletado, interruptores y componentes AJAX.	El JAR se ofrece como biblioteca global de Liberty.
JPA / EclipseLink	Mapeo entre objetos Java y tablas PostgreSQL.	La aplicación no crea ni modifica el esquema: <code>database.action=none</code> .
EJB	Implementa los DAO como servicios <code>@Stateless</code> .	El contenedor gestiona transacciones y ciclo de vida.
PostgreSQL	Persistencia de datos clínicos.	Se accede por el recurso JNDI <code>jdbc/GalenoSV</code> .
Open Liberty	Servidor de aplicaciones.	Ejecuta el WAR y aporta las APIs de Jakarta.
Maven	Compilación, pruebas, empaquetado WAR y modo de desarrollo.	El comando principal es <code>mvn liberty:dev</code> .

No confundir: Maven prepara y empaqueta la aplicación; Open Liberty la ejecuta. NetBeans es el entorno de desarrollo, pero no necesita administrar el despliegue cuando se usa `mvn liberty:dev` desde la terminal.

3. Arquitectura del proyecto

```

        Usuario
    ↓ evento en XHTML / PrimeFaces
boundary.jsf · Modelo de vista
    ↓ invoca operaciones
    control · DAO / EJB
    ↓ usa EntityManager y JPA
    entity · Entidades
    ↓ mapeo relacional
        PostgreSQL

```

entity : representación de los datos

Cada entidad representa una tabla. Sus anotaciones indican tabla, columnas, clave primaria y relaciones. Las entidades no controlan botones ni muestran mensajes; su responsabilidad es describir datos y asociaciones.

control : acceso a datos

Los DAO contienen la comunicación con JPA. `DefaultDAO<T>` reúne operaciones repetidas como buscar, listar, contar, crear, modificar y eliminar. Los DAO concretos indican la entidad que administran y agregan consultas particulares cuando hacen falta.

boundary.jsf : estado y comportamiento de la vista

Los modelos JSF reciben los eventos de las páginas, mantienen el registro seleccionado y deciden si el formulario está creando, modificando o en reposo. Se llaman *Model* porque ese es el nombre usado en el flujo actual del curso, aunque técnicamente actúan como modelos de vista o backing beans.

boundary.convertidores : traducción entre representaciones

Los convertidores resuelven diferencias entre lo que maneja la interfaz o JPA y el tipo Java esperado. Son parte de la frontera de la aplicación, por eso se ubicaron debajo de `boundary`.

webapp : presentación

Las páginas XHTML definen tablas, formularios y acciones. No contienen consultas SQL ni crean un `EntityManager`; delegan ese trabajo a los modelos y DAO.

Pregunta: ¿Boundary y Control son lo mismo?

No. Boundary atiende la interacción con la interfaz. Control ejecuta la lógica de acceso a datos. Separarlos evita que la página conozca detalles de persistencia.

4. Recorrido de una operación CRUD

El mismo patrón se repite en las páginas, con variaciones según las relaciones de cada entidad.

1. JSF construye la página y obtiene el bean mediante su nombre, por ejemplo `#{tipoExamenModel}`.
2. `@PostConstruct` inicializa un `LazyDataModel`.
3. PrimeFaces solicita una página de registros.
4. El modelo llama a `DAO.findRange(first, max)` y `DAO.count()`.
5. Al presionar **Nuevo**, el modelo crea una entidad y cambia el estado a `CREAR`.
6. Al guardar, el modelo valida y llama a `DAO.crear(registro)`.
7. El DAO usa `EntityManager.persist`.
8. PrimeFaces actualiza por AJAX la tabla, el detalle y los mensajes.

```
p:commandButton → método del Model → método del DAO → EntityManager → PostgreSQL
PostgreSQL → resultado → Model → actualización AJAX → usuario
```

Estados del formulario

Estado	Significado	Comportamiento esperado
NADA	No existe una edición activa.	Se muestra la tabla y la opción de crear.
CREAR	Se está preparando un registro nuevo.	Se muestra Guardar y Cancelar.
MODIFICAR	Se seleccionó un registro existente.	Se muestran Modificar, Eliminar y Cancelar.

5. Por qué existen clases abstractas y genéricos

El proyecto tiene veinte entidades con operaciones CRUD parecidas. Repetir la misma paginación, selección, persistencia y mensajes en veinte clases aumentaría los errores. La abstracción conserva el comportamiento común y obliga a cada clase concreta a resolver lo que cambia.

`DefaultDAO<T>`

El parámetro `T` representa el tipo de entidad. Por ejemplo:

```
public class TipoExamenDAO extends DefaultDAO<TipoExamen> {
    public TipoExamenDAO() {
        super(TipoExamen.class);
    }
}
```

Así, los métodos heredados reciben y devuelven `TipoExamen`. Esto aporta seguridad de tipos: no se puede enviar accidentalmente una `Persona` a un DAO de exámenes.

`AbstractModel<T>`

Centraliza la tabla diferida, selección, botones y mensajes. Cada modelo concreto proporciona:

- El DAO correspondiente.
- La forma de construir un registro nuevo.
- La forma de obtener y convertir su identificador.
- Las reglas especiales de su pantalla.

Pregunta: ¿No está demasiado factorizado?

La abstracción se limita al CRUD que realmente se repite. Las relaciones, búsquedas y validaciones especiales permanecen en modelos concretos. No se intentó construir un generador universal de pantallas; se reutilizó únicamente el comportamiento común comprobable.

Pregunta: ¿Por qué se eliminaron algunos `@Override`?

Cuando una clase concreta repetía exactamente el comportamiento heredado, la sobrescritura no aportaba nada. Se conservaron los `@Override` donde la entidad necesita crear su UUID, sincronizar relaciones, personalizar una búsqueda o cambiar el comportamiento del CRUD.

6. Modelo de datos y flujo funcional

El flujo correcto no depende únicamente del orden visual del menú. Depende de qué registros necesita cada relación.

Fase A: configuración

1. **Clínica:** establecimiento donde trabajan o son atendidas las personas.
2. **Rol:** paciente, odontólogo, asistente u otro.
3. **Tipos:** tipos de documento, medios de contacto y exámenes.
4. **Examen:** examen específico, como radiografía panorámica.
5. **Procedimiento:** tratamiento, como limpieza o extracción.
6. **Pasos:** actividades que forman un procedimiento, su secuencia y los exámenes requeridos.

Fase B: registro de personas

1. **Persona:** datos personales.
2. **PersonaRol:** vincula persona, clínica y rol.
3. **Documento:** vincula persona y tipo de documento.
4. **MedioContacto:** vincula persona y tipo de contacto.

Fase C: atención clínica

1. **Consulta:** se abre para una `PersonaRol`.
2. **ConsultaProcedimiento:** registra el procedimiento aplicado dentro de la consulta.
3. **ConsultaProcedimientoPaso:** registra la ejecución de un paso y la persona responsable.
4. **OrdenExamen:** nace desde el paso de procedimiento ejecutado.
5. **ExamenResultado:** almacena resultado, interpretación y ruta del atestado.

Ejemplo narrado:

Se registra a Juan Pérez como persona y se le asigna el rol Paciente en una clínica. Se abre una consulta por dolor en un molar, se agrega el procedimiento de evaluación, se registra el paso radiológico, se crea una orden y finalmente se guarda el resultado de la radiografía panorámica.

Relaciones importantes

Persona → PersonaRol ← Clínica / Rol
Persona → Documento ← TipoDocumento
Persona → MedioContacto ← TipoMedioContacto

Procedimiento → ProcedimientoPaso → ProcedimientoPasoExamen ← Examen
Examen → ExamenTipoExamen ← TipoExamen

PersonaRol → Consulta → ConsultaProcedimiento → ConsultaProcedimientoPaso
ConsultaProcedimientoPaso → OrdenExamen → ExamenResultado

7. Caso clave: Examen y TipoExamen

Primero se distingue el significado

Concepto	Ejemplo	Entidad
Categoría o clasificación	Radiológico	TipoExamen
Examen específico	Radiografía panorámica	Examen
Asociación entre ambos	Radiografía panorámica pertenece a Radiológico	ExamenTipoExamen

Examen ← ExamenTipoExamen → TipoExamen

`ExamenTipoExamen` no es un formulario inventado para la interfaz. Es una entidad real que representa la tabla intermedia y además tiene datos propios: identificador, fecha de creación y observaciones. Por eso se modela explícitamente, en lugar de ocultarla como una relación simple.

Justificación del genérico del modelo

```
public class ExamenTipoExamenModel
    extends AbstractModel<ExamenTipoExamen>
    implements Serializable {
}
```

Esta declaración es correcta en GalenoSv porque la responsabilidad principal del modelo es crear, listar, modificar y eliminar registros de la relación `ExamenTipoExamen`. Los demás tipos de la clase confirman esa decisión:

```
ExamenTipoExamenDAO dao;
DefaultDAO<ExamenTipoExamen> getDao();
LazyDataModel<ExamenTipoExamen> modelo;
ExamenTipoExamen nuevoRegistro();
```

La búsqueda del elemento que se asociará sí trabaja con `TipoExamen`:

```
TipoExamenDAO tipoExamenDAO;
TipoExamen tipoExamenSeleccionado;
```

Cómo explicarlo al ingeniero:

El ejemplo de clase podía usar `AbstractModel<TipoExamen>` si su única responsabilidad era buscar o seleccionar tipos. Al integrarlo con nuestra base, el modelo pasó a administrar la entidad de enlace completa. Conservamos la búsqueda con `TipoExamen`, pero el CRUD pertenece a `ExamenTipoExamen`.

¿Qué pasaría si se cambiara a `AbstractModel<TipoExamen>`?

No sería un simple cambio de nombre. El contrato heredado exigiría que `getDao()`, `nuevoRegistro()`, `selectionHandler()` y la tabla trabajaran con `TipoExamen`. El DAO actual trabaja con `ExamenTipoExamen`; por lo tanto, los tipos serían incompatibles o se perdería el CRUD de la relación.

Respuesta corta para la exposición:

El tipo genérico expresa qué entidad administra el modelo. Como este modelo persiste la relación, debe usar `ExamenTipoExamen`. El objeto `TipoExamen` se usa como opción seleccionable dentro de esa relación.

8. Por qué `@Dependent` en el modelo de detalle

`ExamenModel` es `@ViewScoped`: mantiene el examen seleccionado mientras el usuario permanece en la vista. Dentro de él se inyecta `ExamenTipoExamenModel`, que es `@Dependent`.

```
@Named("examenModel")
@ViewScoped
public class ExamenModel implements Serializable {
    @Inject
    ExamenTipoExamenModel examenTipoExamenModel;
}

@Named("examenTipoExamenModel")
@Dependent
public class ExamenTipoExamenModel { ... }
```

Al ser dependiente, el modelo secundario vive asociado al objeto que lo recibe. Esto permite que el detalle de tipos acompañe al examen seleccionado y evita administrar otra vista independiente para una relación que se muestra dentro de `Examen.xhtml`.

Pregunta: ¿Por qué hay 20 modelos y solo 19 páginas?

Porque `ExamenTipoExamenModel` no representa una página separada. Es el modelo del detalle incluido en la pestaña Tipos de examen dentro de `Examen.xhtml`.

9. Pestaña, diálogo y autocompletado

`Examen.xhtml` usa un `p:tabView` con dos contextos:

- **Datos del examen:** nombre, activo y observaciones.
- **Tipos de examen:** asociaciones del examen seleccionado.

La segunda pestaña se deshabilita mientras no exista un examen seleccionado. Esto evita intentar relacionar tipos con un examen que todavía no fue guardado.

Para elegir un tipo se abre un `p:dialog` con `p:autoComplete`. El autocompletado llama a `TipoExamenDAO.findByNombreLike`, respaldado por una `NamedQuery`. La búsqueda comienza con tres caracteres para evitar consultas amplias por cada tecla.

¿TabView o Wizard?

Componente	Cuándo conviene	Decisión actual
<code>p:tabView</code>	Cuando son secciones relacionadas y se puede volver libremente entre ellas.	Adecuado para editar el examen y luego sus asociaciones.
<code>p:wizard</code>	Cuando existe un proceso obligatorio paso a paso con validación antes de avanzar.	Sería útil si el alta completa exigiera una secuencia rígida.

No se eligió `wizard` porque la pantalla actual administra un maestro y su detalle, no un trámite lineal que deba completarse una sola vez.

10. Los dos convertidores no hacen lo mismo

`TipoExamenConverter` : convertidor de JSF

En HTML, el valor seleccionado llega como texto. El autocompletado necesita recuperar un objeto `TipoExamen`. El convertidor transforma UUID en entidad y entidad en UUID:

```
navegador envía String UUID
      ↓ getAsObject
TipoExamenDAO.find(UUID)
      ↓
objeto TipoExamen
```

`UUIDConverter` : convertidor de JPA

Implementa `AttributeConverter<UUID, UUID>` para declarar explícitamente la conversión de identificadores UUID al tipo UUID de PostgreSQL. No participa en componentes JSF ni busca entidades.

Pregunta: ¿Por qué están en `boundary.convertores` ?

Se agruparon siguiendo la organización del proyecto base. NetBeans puede mostrar `boundary`, `boundary.jsf` y `boundary.convertores` como paquetes separados si usa vista plana; físicamente los dos últimos están debajo de la carpeta `boundary`.

11. Persistencia y decisiones de las entidades

UUID como identificador

Los registros generan UUID en Java antes de persistirse. Esto evita depender de una secuencia numérica central y coincide con las columnas UUID de PostgreSQL.

`OffsetDateTime` para fecha y hora

Los campos temporales usan `OffsetDateTime`. Se eliminó el patrón antiguo `Date` más `@Temporal`, porque la API moderna de Java expresa mejor fecha, hora y desplazamiento.

Relaciones diferidas

Las asociaciones se declaran principalmente con `FetchType.LAZY`. Así no se carga todo el grafo de la clínica cada vez que se consulta una fila. Los DAO que necesitan mostrar relaciones pueden usar consultas con `JOIN FETCH`.

La aplicación no altera el esquema

`jakarta.persistence.schema-generation.database.action=none` protege la estructura existente. La base de datos ya está creada y el proyecto se limita a operar sobre ella.

Pregunta: ¿Por qué las entidades y los DAO implementan `Serializable`?

Los modelos con alcance de vista pueden conservarse entre solicitudes y el contenedor puede necesitar serializar su estado. Mantener serializables los objetos participantes evita problemas en ese ciclo de vida y sigue la observación realizada al proyecto.

12. Plantilla e internacionalización

Las páginas usan `/WEB-INF/plantillas/general.xhtml`. La ubicación bajo `WEB-INF` impide abrir la plantilla directamente y obliga a consumirla mediante Facelets.

La plantilla concentra:

- Título y encabezado.
- Menú PrimeFaces agrupado por áreas.
- Zona de mensajes.
- Selector de idioma.
- Contenido y pie de página.

`faces-config.xml` registra el bundle con la variable `msg`. Existen recursos para español, inglés y francés. Las páginas usan claves como `#{msg['boton.guardar']}` en lugar de repetir texto fijo.

Pregunta: ¿Qué ventaja da la plantilla?

Evita copiar navegación, selector de idioma y mensajes en cada página. Un cambio común se realiza una vez y se refleja en todas las vistas.

13. Configuración de PrimeFaces y Liberty

PrimeFaces está declarado en Maven con clasificador `jakarta` y alcance `provided`. Eso significa que el código compila contra PrimeFaces, pero el WAR no necesita empaquetar otra copia porque Liberty lo proporciona como biblioteca global.

```
<dependency>
  <groupId>org.primefaces</groupId>
  <artifactId>primefaces</artifactId>
  <version>15.0.17</version>
  <classifier>jakarta</classifier>
  <scope>provided</scope>
</dependency>
```

El archivo `primefaces-lib.xml` define la biblioteca global y busca `primefaces-15.0.17-jakarta.jar` dentro de `${wlp.install.dir}/lib`. Se cambió el nombre genérico `openliberty.xml` por uno que explica su propósito.

Conexión a la base

`persistence.xml` usa la unidad `clinica_ppi` y solicita el datasource JTA `jdbc/GalenoSV`. La definición del datasource pertenece al servidor Liberty externo; no está duplicada dentro de la aplicación.

Limitación de portabilidad conocida:

En esta computadora funciona porque Liberty ya tiene configurado `jdbc/GalenoSV`. Otra persona que clone el repositorio necesitará configurar el mismo datasource, el controlador PostgreSQL y la biblioteca global de PrimeFaces en su servidor.

14. Decisiones tomadas a partir de revisiones

Observación	Aplicación en GalenoSv	Justificación
Jakarta EE estable	Versión 11.0.0.	Se evita depender de una versión preliminar M1.
Compilar con Java 21	<code>maven-compiler-plugin</code> 3.12.1.	La versión queda reproducible y explícita.
Nombre específico para configDropins	<code>primefaces-lib.xml</code> .	El nombre comunica qué configura.
API temporal moderna	<code>OffsetDateTime</code> sin <code>@Temporal</code> .	Se evita una API heredada y se conserva información temporal adecuada.
DAO serializables	<code>DefaultDAO</code> implementa <code>Serializable</code> .	Los DAO concretos heredan esa capacidad.
Modelo secundario dependiente	<code>ExamenTipoExamenModel</code> <code>@Dependent</code> .	Su ciclo de vida queda ligado al modelo principal.

15. Qué está verificado y qué no debe exagerarse

Verificado en esta fase

- El proyecto compila y genera el WAR con Maven.
- Las 14 pruebas automatizadas terminan sin fallos.
- Las entidades revisadas corresponden a las tablas y columnas de la base existente.
- Las páginas fueron incorporadas a la plantilla y al menú.

- Liberty despliega la aplicación y las páginas comprobadas responden correctamente.
- La relación de examen y tipo de examen funciona dentro de la página de examen.
- No se mantienen clases `Frm` duplicadas junto a los modelos JSF actuales.

Límites honestos

- Las 14 pruebas no cubren todavía los veinte DAO ni todos los recorridos completos de interfaz.
- El datasource y las bibliotecas globales dependen de la configuración del Liberty externo.
- `PROJECT_STAGE` está en `Development`, apropiado para clase, no para producción.
- El plugin de Liberty contiene una ruta de instalación propia de esta computadora.
- Autenticación, autorización y auditoría no forman parte de la fase actual.

Forma correcta de decirlo:

“El proyecto está funcional y coherente para el alcance implementado. Compila, despliega y cuenta con pruebas enfocadas. Aún puede ampliarse la cobertura y preparar una configuración portable o de producción.”

16. Demostración sugerida

Este recorrido permite enseñar el patrón sin intentar abrir las diecinueve páginas.

1. **Mostrar el menú:** explicar los grupos Catálogos, Clínica, Personas, Procedimientos y Atención.
2. **Abrir Tipo de examen:** enseñar un CRUD sencillo y el uso del `toggleSwitch`.
3. **Abrir Examen:** seleccionar o crear una radiografía panorámica.
4. **Abrir la pestaña Tipos:** mostrar la relación interna, el diálogo y el autocompletado.
5. **Mostrar Persona:** explicar que los campos corresponden a su entidad y no al formulario genérico inicial.
6. **Explicar PersonaRol:** demostrar por qué persona, clínica y rol son registros separados.
7. **Recorrer Atención:** narrar consulta → procedimiento → paso → orden → resultado.
8. **Mostrar código:** abrir una entidad, su DAO, su modelo y su XHTML para seguir una operación de extremo a extremo.
9. **Cerrar con Maven:** mencionar compilación, WAR y pruebas.

Guion de cinco minutos

Minuto 1: GalenoSv es una adaptación del proyecto base para gestión odontológica y usa la misma separación arquitectónica.

Minuto 2: La página llama al modelo, el modelo al DAO, el DAO a JPA y JPA a PostgreSQL.

Minuto 3: `AbstractModel` y `DefaultDAO` reutilizan CRUD; las clases concretas conservan las reglas específicas.

Minuto 4: explicar `Examen ← ExamenTipoExamen → TipoExamen` y por qué el genérico es la entidad intermedia.

Minuto 5: resumir flujo clínico, internacionalización, Liberty y pruebas.

17. Preguntas probables y respuestas

¿Por qué usar DAO si JPA ya tiene EntityManager?

Porque el DAO concentra las operaciones de persistencia y evita que la interfaz dependa directamente de JPA. También ofrece un lugar único para consultas específicas y manejo coherente de errores.

¿Por qué no colocar todo en el modelo JSF?

Porque mezclaría estado visual, navegación y persistencia. Separarlo facilita probar los DAO y cambiar la interfaz sin reescribir el acceso a datos.

¿Qué diferencia hay entre Examen y TipoExamen?

TipoExamen clasifica; Examen representa el estudio concreto. “Radiológico” es tipo y “Radiografía panorámica” es examen.

¿Por qué existe una entidad intermedia?

Porque la relación tiene identidad y datos propios, como fecha y observaciones. Además permite que la asociación evolucione sin cargar esos datos en ninguna de las dos entidades principales.

¿Por qué usar un modelo @Dependent ?

Porque el modelo de la relación es un detalle del examen seleccionado y debe acompañar el ciclo de vida del modelo principal.

¿Por qué @ViewScoped y no @RequestScoped ?

Una pantalla JSF realiza varias solicitudes AJAX. El alcance de vista conserva selección y estado CRUD durante esas solicitudes.

¿Por qué usar LazyDataModel ?

Para pedir únicamente el rango visible de filas y el total, en lugar de cargar toda la tabla en memoria.

¿Por qué PrimeFaces tiene alcance provided ?

Porque la biblioteca se entrega desde Liberty como biblioteca global. Empaquetarla también en el WAR duplicaría clases y podría provocar conflictos de versión.

¿Por qué no generar automáticamente la base con JPA?

Porque el esquema ya existe y es la fuente de verdad. La acción `none` evita modificaciones accidentales durante el despliegue.

¿Por qué OffsetDateTime ?

Es la API moderna para fecha y hora con desplazamiento; reemplaza el uso heredado de `Date` y `@Temporal`.

¿La aplicación ya está lista para producción?

Está lista para el alcance académico actual, no para producción. Antes harían falta seguridad, auditoría, más pruebas, configuración portable y cambiar el modo Development.

¿Por qué NetBeans muestra tres paquetes?

Java considera `boundary`, `boundary.jsf` y `boundary.conversores` paquetes distintos. La vista plana los muestra como hermanos, aunque físicamente los subdirectorios estén dentro de `boundary`.

¿Se abandonó el patrón del proyecto base?

No. Se mantuvieron entidad, DAO, modelo JSF, plantilla y página. Las diferencias responden al dominio odontológico y a relaciones que el ejemplo original no necesitaba resolver completamente.

18. Lista rápida antes de exponer

- Levantar PostgreSQL con Docker.
- Confirmar el datasource de Liberty.
- Ejecutar `mvn liberty:dev`.
- Abrir una página antes de comenzar.
- Tener tipos y exámenes de ejemplo.
- Elegir un registro que muestre la relación.
- Evitar editar datos importantes durante la demostración.
- Tener este PDF abierto como respaldo.
- Recordar que son 20 entidades y 19 páginas.
- Explicar el flujo, no leer cada clase.

19. Resumen final para memorizar

Arquitectura: XHTML → Model → DAO → Entity → PostgreSQL.

Reutilización: `AbstractModel<T>` y `DefaultDAO<T>` reúnen el CRUD común.

Especialización: cada modelo y DAO concreto conserva las reglas particulares.

Caso clave: `ExamenTipoExamenModel` usa `ExamenTipoExamen` porque administra la relación; usa `TipoExamen` únicamente como selección.

Flujo clínico: configuración → persona y rol → consulta → procedimiento → paso → orden → resultado.

Estado: funcional para la fase académica actual, compilado, desplegado y con 14 pruebas aprobadas.

Documento elaborado a partir del estado real del código de GalenoSv en la fecha de corte indicada. Las decisiones descritas corresponden a la implementación actual y al alcance trabajado en clase.